

Compiler-2 보고서

I. 프로그램 설명

프로그램(pracmain.c)은 initialize() -> yyparse() -> print_ast() 순으로 진행된다. 첫 번째, initialize()에서는 기본적인 type(int,float,char,void,string) table과 type의 size를 정의한다. 두 번째 단계인 yyparse()에서는 lex로부터 token을 하나씩 받아와 shift/reduce 작업을 수행하며 문법을 검사한다. 이때, root 노드인 N_PROGRAM 하위에 여러 종류의 노드, symbol table, type table, syntax tree를 연결하여 이후 semantic 분석을 위한 준비를 하도록 구현하였다.

II. 프로그램 수행 도중 발생한 오류 및 해결 방법

① 토큰 값(yylval)을 int 타입에서 YYSTYPE(long) 타입으로 확장하였는데 error가 발생하였다.

- 해결 방법: 기존의 프로그램에 존재하던 yylval과 중복으로 선언된 것을 확인하고 수정을 하였다.

② 프로그램이 malloc.c 파일을 찾지 못한다는 오류와 함께 segment fault가 발생하였다.

- 해결 방법: gdb를 통한 분석 결과, 어떠한 선언문도 A_ID가 제대로 생성이 되지 않는 것을 확인하였다. 이는 lex 프로그램 중 yylval 값을 저장하고 리턴하는 과정이 존재하지 않아 생기는 문제였다. 따라서 lex 프로그램에 INTEGER_CONSTANT, FLOAT_CONSTANT, IDENTIFIER, TYPE_IDENTIFIER, STRING_LITERAL, CHARACTER_CONSTAT의 yylval 값을 경우에 따라 저장하고 리턴하는 코드를 추가하였다. 이때 checkIdentifier 함수로 TYPE_IDENTIFIER와 IDENTIFIER를 구분하도록 하였다.

③ 함수, 구조체, 반복문, 조건문 등 간단한 declaration 선언 외에는 모두 segment fault가 발생하였다.

- 해결 방법: gdb를 통한 분석 결과, setDeclaratorElementType 함수에서 type을 설정하는 부분의 일부 분이 실행되지 않은 것을 확인하였다. prac.y 파일에서 func.c에 정의된 함수들의 프로토타입이 일부 선언되지 않아 발생한 문제라는 것을 파악하였다. 따라서, 작성되지 않은 함수들의 프로토타입을 선언해 주었다.

Ⅲ. Test Case

1.

[입력]:

```
$ cat ex.c
struct st { int a; };

int main(){
    union st difftype;
}
```

[출력]:

```
: $ ./a.out < ex.c
line 4: syntax error: illegal referencing struct or union identifier st near difftype
line 5: syntax error: incomplete forward reference for identifier st near }
```

[설명]: sturct_type(type_specifier)으로 하나의 구조체를 선언하였는데, 같은 IDENTIFIER이지만 struct_type이 아닌 union_type을 선언하였다. 따라서, struct identifier(st)의 type_table을 가져올 때 에러가 발생한다. getTypeOfStructOrEnumRefIdentifier에서 type이 일치하는지 확인할 때, id->type->kind는 T_STRUCT이지만, k는 T_UNION이어서 일치하지 않는다. 따라서 새로운 union 타입을 생성하게 되는데, 프로그램 끝에서 전방 참조를 할 때 id->kind가 ID_FIELD 여서 오류가 발생한다. 또한, t->field도 존재하지 않는다.

2.

[입력]:

```
struct st { int a;};
union un;
enum e {A,B,C};

int main(){
    struct st new1;
    struct st new2={1};
}
```

[출력]:

```
line 9: syntax error: incomplete forward reference for identifier un at end
```

[설명]: 프로그램 끝에서 전방 참조를 할 때, union 타입의 un이 완전한 정의 없이 선언만 되어 있어 오류가 발생한다.

3.

[입력]:

```
struct st { int a; struct st ss=(2); };
union un { float b; union un abc; };
enum e {A,B,C};

int main(){
    struct st new1;
    struct st new2={1};
}
```

[출력]:

```
line 1: syntax error near =
```

[설명]: struct는 정의하는 부분에서 초기화 식이 나올 수 없기 때문에 오류가 발생한다.

4.

[입력]:

```
struct s1;
struct { char a; struct s1 *re; };
struct s2 { int b; };
union u1;
union { char c; int ccccc; } u2;
union u3 { float c; union u1 abc; };
enum e {A,B,C};

int main(){
    struct s1 { float a; };
    struct s1 new1={1.23};
    struct s2 new2={1};

    u2={'a', 1};
    union u1 { int u1; int u2; int u3; };
}
```

[출력]:

```
line 14: syntax error near {
```

[설명]: unary_expression ASSIGN assignment_expression rule에서 assignment에 '{ }'와 관련된 문법이 존재하지 않아 오류가 발생한다.

5.

[입력]:

```
struct s1;
struct { char a; struct s1 *re; };
struct s2 { int b; };

int main(){
    struct s1 new1={1.23};
}

struct s1 { float a; };
```

[출력]:

```
===== syntax tree =====
N_PROGRAM (0,0)
| (ID="") TYPE:8f542910 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | TYPE
| | | STRUCT
| | | | FIELD
| | | | | (ID="a") TYPE:8f53d300 KIND:FIELD SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | TYPE
| | | | | (float)
| (ID="") TYPE:8f542a10 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | TYPE
| | | STRUCT
| | | | FIELD
| | | | | (ID="a") TYPE:8f53d410 KIND:FIELD SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | TYPE
| | | | | (char 1)
| | | | | (ID="re") TYPE:8f542ae0 KIND:FIELD SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | TYPE
| | | | | | POINTER
| | | | | | ELEMENT_TYPE
| | | | | | (DONE:8f542910)
| (ID="") TYPE:8f542c50 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | TYPE
| | | STRUCT
| | | | FIELD
| | | | | (ID="b") TYPE:8f53d2f0 KIND:FIELD SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | TYPE
| | | | | (int)
| (ID="main") TYPE:8f542e00 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | | TYPE
| | | | | (int)
| | | | BODY
| | | | | N_STMT_COMPOUND (0,0)
| | | | | | (ID="new1") TYPE:8f542910 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | | (DONE:8f542910)
| | | | | | | INIT
| | | | | | | N_INIT_LIST (0,0)
| | | | | | | | N_INIT_LIST_ONE (0,0)
| | | | | | | | | N_EXP_FLOAT_CONST (0,0)
| | | | | | | | | | 1.23
| | | | | | | | N_INIT_LIST_NIL (0,0)
| | | | | | N_STMT_LIST_NIL (0,0)
| (ID="") TYPE:8f542910 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | TYPE
| | | (DONE:8f542910)
```

[설명]: struct s1은 맨 처음에 선언만 되어 있었지만, 나중에 정의부가 나타나 정상적으로 정의된다. 이때 syntax tree에서 확인할 수 있는 것은 프로그램 첫 부분에 선언만 하고, 정의를 프로그램 마지막에 하더라도 선언하는 위치에 정의부에 대한 정보가 같이 나타난다. 다른 struct 정의 부분 역시도 작성한 순서대로 syntax tree에 나타난다. 또한, 함수의 compound_statement가 body 부분에 node로 연결되는 것을 확인할 수 있다. 그리고 level값이 하나 증가한 상태에서 선언된 struct s1 new1의 선언 부분에서 expression이 node로 연결되어 있고, 다시 FLOAT_CONSTANT node 하위에 실제 float 값(float string)이 존재하는 것을 확인할 수 있다.

6.

[입력]:

```
1 union u1;
2 union u2 { float b1; union u1 b2; };
3 union u3 { char a1; char a2; int a3; } un2;
4
5 main(){
6     char c1='c';
7     char c2='d';
8     int i1=0;
9     un2={c1,c2,i1};
10    un2.a2=i1;
11    un2.a2=c2;
12 }
13
14 union u1 { int k;};
```

[출력]:

```
line 9: syntax error near {
```

[설명]: union의 값을 중괄호를 통해 입력하는 것은 문법적으로 정의가 되어 있지 않기 때문에 오류가 발생한다.

7.

[입력]:

```
union u1;
union u2 { float b1; union u1 b2; };
union u3 { char a1; char a2; int a3; } un2;

main(){
    char c2='d';
    int i1=0;

    un2.a2=i1;
    un2.a2=c2;
}

union u1 { int k;};
```

[출력]:

```

===== syntax tree =====
N_PROGRAM (0,0)
| (ID="") TYPE:80443910 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | TYPE
| | | UNION
| | | | FIELD
| | | | | (ID="k") TYPE:8043e2f0 KIND:FIELD SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | | (int)
| (ID="") TYPE:80443a80 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | TYPE
| | | UNION
| | | | FIELD
| | | | | (ID="b1") TYPE:8043e380 KIND:FIELD SPEC=MALL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | | (float)
| | | | | (ID="b2") TYPE:80443910 KIND:FIELD SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | | (DONE:80443010)
| (ID="un2") TYPE:80443c80 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | TYPE
| | | UNION
| | | | FIELD
| | | | | (ID="a1") TYPE:8043e410 KIND:FIELD SPEC=MALL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | | (char 1)
| | | | | (ID="a2") TYPE:8043e410 KIND:FIELD SPEC=MALL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | | (char 1)
| | | | | (ID="a3") TYPE:8043e2f0 KIND:FIELD SPEC=MALL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | | (int)
| (ID="main") TYPE:80443f10 KIND:FUNC SPEC=MALL LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | | TYPE
| | | | | | (int)
| | | | | BODY
| | | | | | N_STMT_COMPOUND (0,0)
| | | | | | | (ID="c2") TYPE:8043e410 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=0
| | | | | | | | TYPE
| | | | | | | | | (char 1)
| | | | | | | | | INIT
| | | | | | | | | | N_INIT_LIST_ONE (0,0)
| | | | | | | | | | | N_EXP_CHAR_CONST (0,0)
| | | | | | | | | | | | 100
| | | | | | | | | | (ID="l1") TYPE:8043e2f0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=0
| | | | | | | | | | | TYPE
| | | | | | | | | | | | (int)
| | | | | | | | | | | | INIT
| | | | | | | | | | | | | N_INIT_LIST_ONE (0,0)
| | | | | | | | | | | | | | N_EXP_INT_CONST (0,0)
| | | | | | | | | | | | | | | 0
| | | | | | | | | | N_STMT_LIST (0,0)
| | | | | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | | | | N_EXP_ASSIGN (0,0)
| | | | | | | | | | | | | N_EXP_STRUCT (0,0)
| | | | | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | | | | (ID="un2") TYPE:80443c80 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | | | | | | | | | | | | | | | a2
| | | | | | | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | | | | | | (ID="l1") TYPE:8043e2f0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=0
| | | | | | | | | | | | | | | | N_STMT_LIST (0,0)
| | | | | | | | | | | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | | | | | | | | | | N_EXP_ASSIGN (0,0)
| | | | | | | | | | | | | | | | | | N_EXP_STRUCT (0,0)
| | | | | | | | | | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | | | | | | | | (ID="un2") TYPE:80443c80 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | | | | | | | | | | | | | | | | | | | none
| | | | | | | | | | | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | | | | | | | | | | (ID="c2") TYPE:8043e410 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=0
| | | | | | | | | | | | | | | | | | | | | N_STMT_LIST_NIL (0,0)
| (ID="") TYPE:80443910 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | TYPE
| | | (DONE:80443010)

```

[설명]: 이전의 결과들과 같이 선언된 union 순서대로 정의가 되어 있고, 같은 level 0에 존재하는 것을 syntax tree를 통해 확인할 수 있다. field의 name, type도 제대로 출력되고, 각 scope(level) 값 역시도 중괄호 시작과 끝 부분에서 증가하고 감소하여 적절한 값이 입력되었다. main 함수에서는 char_type인 un2.a2에 int_type i1 값을 입력했고, 다른 타입이지만 노드와 symbol table이 연결되었다. un2.none이라는 union에 존재하지 않는 field도 노드와 symbol table이 연결되어 정상 출력된 것을 확인하였다.

8.

[입력]:

```
1 int a;  
2 char a[3];  
3 float a;  
4 int a(char a[]);  
5 struct {int s; } a;  
6 union {int s; } a;  
7 enum {A} a;
```

[출력]:

```
line 2: syntax error: redeclaration of identifier a near ;  
line 3: syntax error: redeclaration of identifier a near ;  
line 4: syntax error: redeclaration of identifier a near ;  
line 5: syntax error: redeclaration of identifier a near ;  
line 6: syntax error: redeclaration of identifier a near ;  
line 7: syntax error: redeclaration of identifier a near ;
```

[설명]: 같은 identifier(a)를 여러 다른 타입으로 재선언해서 오류가 발생하였다.

9.

[입력]:

```
1 int a;  
2 char b[3];  
3 int d(char a[]);  
4 hamsoo(int a, int b){  
5     int d,i;  
6     for(i=0; ;){  
7         float a;  
8     }  
9 }
```

[출력]:


```

===== syntax tree =====
N_PROGRAM (0,0)
| (ID="a") TYPE:a7be42f0 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | TYPE
| | | (int)
| (ID="b") TYPE:a7be9a50 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | TYPE
| | | ARRAY
| | | | INDEX
| | | | | N_EXP_INT_CONST (0,0)
| | | | | 3
| (none)
| | | ELEMENT_TYPE
| | | | (char 1)
| (ID="d") TYPE:a7be9b00 KIND:FUNC SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | | (ID="a") TYPE:a7be9b90 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | | ARRAY
| | | | | | | | INDEX
| | | | | | | | (none)
| | | | | | | | ELEMENT_TYPE
| | | | | | | | (char 1)
| | | | | | TYPE
| | | | | | (int)
| (ID="hansoo") TYPE:a7be9d60 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | | (ID="a") TYPE:a7be42f0 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | (int)
| | | | | (ID="b") TYPE:a7be42f0 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | (int)
| | | | TYPE
| | | | (int)
| | | | BODY
| | | | | N_STMT_COMPOUND (0,0)
| | | | | | (ID="d") TYPE:a7be42f0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | | (int)
| | | | | | (ID="l") TYPE:a7be42f0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=0
| | | | | | | TYPE
| | | | | | | (int)
| | | | | | N_STMT_LIST (0,0)
| | | | | | | N_STMT_FOR (0,0)
| | | | | | | | N_FOR_EXP (0,0)
| | | | | | | | | N_EXP_ASSIGN (0,0)
| | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | (ID="l") TYPE:a7be42f0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=0
| | | | | | | | | | | N_EXP_INT_CONST (0,0)
| | | | | | | | | | | 0
| | | | | | | N_STMT_COMPOUND (0,0)
| | | | | | | | (ID="a") TYPE:a7be4380 KIND:VAR SPEC=AUTO LEV=2 VAL=0 ADDR=0
| | | | | | | | TYPE
| | | | | | | | | (float)
| | | | | | N_STMT_LIST_NIL (0,0)
| | | | N_STMT_LIST_NIL (0,0)

```

[설명]: 같은 identifier여도 다른 level이면 선언이 가능하다. 이 때, 각 scope에 따라 level이 증가하고 감소하는 것을 syntax tree를 통해 확인할 수 있다. 또한, type_specifier가 없는 hansoo(함수)는 default로 int_type으로 설정되고 storage_class가 없는 변수들은 S_AUTO로 설정된다. for문 안에 있는 a는 초기화된 expression이 없기 때문에 N_STMT_LIST_NIL 노드가 연결된 것을 확인할 수 있다.

10.

[입력]:

```
1 hamsoo(int a, int b){
2     a=c;
3     hamsoo_ref(2,3,4,5);
4     a = e2.A;
5     b = st -> s;
6 }
```

[출력]:

```
line 2: syntax error: undefined identifier c near c
line 3: syntax error: undefined identifier hamsoo_ref near hamsoo_ref
line 4: syntax error: undefined identifier e2 near e2
line 5: syntax error: undefined identifier st near st
```

[설명]: 선언이 되지 않은 변수, 함수, enum, struct의 identifier를 입력해서 해당 identifier들을 찾지 못한다는 error message가 출력되었다.

11.

[입력]:

```
1 void hamsoo(void a);
2 void hamso(static b);
3 void hams(auto c);
4 void ham(typedef int abcde);
5 typedef int INT;
6 void ha(INT d);
7 h(int a, float b[], char**);
```

[출력]:

```
line 1: syntax error: illegal type specifier in formal parameter near )
line 2: syntax error: illegal type specifier in formal parameter near )
line 3: syntax error: illegal type specifier in formal parameter near )
line 4: syntax error: illegal type specifier in formal parameter near )
line 7: syntax error: empty parameter name near ;
line 7: syntax error near ;
```

[설명]: 함수의 parameter에는 storage_class가 포함될 수 없다. 따라서 void, static, auto, typedef는 파라미터 오류가 발생하게 된다. 하지만 typedef를 선언하고 새로 생성된 타입을 사용하는 것은 가능하다. 마지막 오류는 함수의 프로토타입(선언부)을 정의부와 구별하지 못한 것으로 예상된다.

12.

[입력]:

```
void h(int a, float b[], char **);
```

[출력]:

```

***** syntax tree *****
N_PROGRAM (0,0)
| (ID="h") TYPE:55568c10 KIND:FUNC SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | | (ID="a") TYPE:555632f0 KIND:PARM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | | (int)
| | | | | (ID="b") TYPE:55568aa0 KIND:PARM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | | ARRAY
| | | | | | | | INDEX
| | | | | | | | | (none)
| | | | | | | | ELEMENT_TYPE
| | | | | | | | | (float)
| | | | | (ID="") TYPE:55568b80 KIND:PARM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | | POINTER
| | | | | | | | ELEMENT_TYPE
| | | | | | | | | (char 1)
| | | | | TYPE
| | | | | (void)[Inferior 1 (process 6692) exited normally]

```

[설명]: 함수의 IDENTIFIER 앞에 type_specifier를 추가하면 함수의 프로토타입임을 인식하고, 파라미터들을 적절한 level, kind, type가 설정된 것을 확인할 수 있다. abstract declarator도 잘 인식이 된다.

13.

[입력]:

```

1 typedef int INT;
2
3 int main(){
4     struct {int a;int b;} A;
5     sizeof(A);
6     sizeof(static int);
7     sizeof(auto int);
8     sizeof(typedef int INT);
9     sizeof(INT);
10 }

```

[출력]:

```

line 6: syntax error: illegal storage class in type specifiers near )
line 7: syntax error: illegal storage class in type specifiers near )
line 8: syntax error: illegal storage class in type specifiers near )

```

[설명]: type_name에 storage_class가 있으면 error message를 출력하게 된다. 따라서, storage_class인 static, auto, typedef가 type_name으로 reduce될 때 해당 메시지가 출력된다. 미리 선언된 struct(A)와 typedef(INT)로 미리 선언하였던 새로운 타입은 사용 가능하다.

14.

[입력]:

```
1 typedef int INT;  
2  
3 int main(){  
4     int a;  
5     (static int)a;  
6     (auto int[10])a;  
7     (typedef int)a;  
8     (float)a;  
9     (float x)a;  
10    (float[10])a;  
11    (INT)a;  
12 }
```

[출력]:

```
line 5: syntax error: illegal storage class in type specifiers near )  
line 6: syntax error: illegal storage class in type specifiers near )  
line 7: syntax error: illegal storage class in type specifiers near )  
line 9: syntax error near x
```

[설명]: type_name에 storage_class가 있으면 error message를 출력하게 된다. 따라서, storage_class인 static, auto, typedef가 type_name으로 reduce될 때 해당 메시지가 출력된다. 그리고 decaration_specifiers 옆에 identifier가 존재하는 none abstract_declarator도 오류가 발생한다. 기존에 존재하는 type은 정상적으로 처리되고 float[10]와 같은 abstract_declarator도 가능하다.

15.

[입력]:

```
1 int x;  
2  
3 int main(){  
4     void aaa=x();  
5 }
```

[출력]:

```

===== syntax tree =====
N_PROGRAM (0,0)
| (ID="x") TYPE:664122f0 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | TYPE
| | | (int)
| (ID="main") TYPE:66417a10 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | TYPE
| | | | | (int)
| | | | BODY
| | | | | N_STMT_COMPOUND (0,0)
| | | | | | (ID="aaa") TYPE:664124a0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | | (void)| | | | | | INIT
| | | | | | | N_INIT_LIST_ONE (0,0)
| | | | | | | N_EXP_FUNCTION_CALL (0,0)
| | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | (ID="x") TYPE:664122f0 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | | | | | | | N_ARG_LIST_NIL (0,0)
| | | | | N_STMT_LIST_NIL (0,0)

```

[설명]: declaration으로 생성된 IDENTIFIER x를 함수라고 생각하여 aaa를 초기화하고 있다. syntax tree 확인 결과 function_call을 통해 함수라고 판단하였음을 알 수 있다. 하지만, 실제 x는 함수가 아니고, 프로토타입 선언된 것이 아니기 때문에 적절한 결과는 아니다.

16.

[입력]:

```

struct a { int k; };
void hamsoo(int [], char **, struct a);

char b,c;
void hamsoo(int d, char **b, struct a c){
    if(d==c->k) d=0;
}

```

[출력]:

```

line 5: syntax error: conflicting parm type in prototype function hamsoo near {

```

[설명]: 함수의 선언부와 정의부의 파라미터의 종류가 달라 오류가 발생한다. '[]'와 'd'의 타입도 다르지만, '**'와 '**b'가 갖는 타입도 다르다고 판단한다.

17.

[입력]:

```
struct a { int k; };
void hamsoo(int [], char **, struct a);

char b,c,d[3];
void hamsoo(int d[3], char **b, struct a c){
    if(d==c->k) d=0;
}
```

[출력]:

```
line 5: syntax error: conflicting parm type in prototype function hamsoo near {
```

[설명]: int []와 int d[3]은 같은 배열 타입이지만, d[3]은 expr 부분에 3이라는 수식(syntax tree)를 갖고 있다. 따라서, 이 부분이 함수의 선언부와 정의부가 다르다고 판단하는 부분일 것으로 예상된다.

18.

[입력]:

```
1 struct l { int k; };
2 void hamsoo(int a, char **b, struct l c);
3 void hamsoo(int a, char **b, struct l c){
4     int d;
5     if(d==c->k) d=0;
6 }
7
```

[출력]:

```

##### syntax tree #####
N_PROGRAM (0,0)
| (ID="") TYPE:6741b960 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | TYPE
| | | STRUCT
| | | | FIELD
| | | | | (ID="h") TYPE:674162f0 KIND:FIELD SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | TYPE
| | | | | | (int)
| | (ID="hansoo") TYPE:6741bd40 KIND:FUNC SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | | (ID="a") TYPE:674162f0 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | TYPE
| | | | | | (int)
| | | | | (ID="b") TYPE:6741bbe0 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | TYPE
| | | | | | POINTER
| | | | | | | ELEMENT_TYPE
| | | | | | | | POINTER
| | | | | | | | | ELEMENT_TYPE
| | | | | | | | | | (char 1)
| | | | | (ID="c") TYPE:6741b960 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | TYPE
| | | | | | (DONE:6741b960)
| | | | | TYPE
| | | | | (void) | (ID="hansoo") TYPE:6741c820 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | TYPE
| | | | FUNCTION
| | | | | PARAMETER
| | | | | | (ID="a") TYPE:674162f0 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | | (int)
| | | | | | (ID="b") TYPE:6741bec0 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | | POINTER
| | | | | | | | ELEMENT_TYPE
| | | | | | | | | POINTER
| | | | | | | | | | ELEMENT_TYPE
| | | | | | | | | | | (char 1)
| | | | | (ID="c") TYPE:6741b960 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | TYPE
| | | | | | (DONE:6741b960)
| | | | | TYPE
| | | | | | (void) | | | BODY
| | | | | | N_STMT_COMPOUND (0,0)
| | | | | | | (ID="d") TYPE:674162f0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=0
| | | | | | | TYPE
| | | | | | | | (int)
| | | | | | | N_STMT_LIST (0,0)
| | | | | | | | N_STMT_IF (0,0)
| | | | | | | | | N_EXP_EQ (0,0)
| | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | (ID="d") TYPE:674162f0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=0
| | | | | | | | | | | N_EXP_ARROW (0,0)
| | | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | | (ID="c") TYPE:6741b960 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | | | | | | | | h
| | | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | | N_EXP_ASSIGN (0,0)
| | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | (ID="d") TYPE:674162f0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=0
| | | | | | | | | | | | | N_EXP_INT_CONST (0,0)
| | | | | | | | | | | | | 0
| | | | | | | | | N_STMT_LIST_NIL (0,0)

```

[설명]: 함수의 파라미터로 int, char **, struct l을 선언부, 정의부 둘 다 동일하게 사용하여 syntax tree가 적절하게 출력되었다.

19.

[입력]:

```
1 struct l { int k; } c;  
2 void hamsoo(int, char **);  
3 void hamsoo(int a, char **b){  
4     int d;  
5     if(d==c->k) d=0;  
6 }
```

[출력]:

```
line 3: syntax error: conflicting parm type in prototype function hamsoo near {
```

[설명]: abstract_declarator 사용이 가능하고 겉으로 보여지는 hamsoo의 타입은 동일하지만, 정의부와 선언부의 parameter가 다르다고 판단하는 것을 확인할 수 있다. 이는 gdb를 통해 확인해 본 결과 char **가 가리키는 주소와 char **b가 가리키는 주소가 달랐다. 결과적으로, 파라미터의 타입이 다르다는 메시지가 발생하게 된다.

20.

[입력]:

```
1 struct l { int k; } c;  
2 void hamsoo(int, char, struct l);  
3 void hamsoo(int , char , struct l){  
4     int d;  
5     if(d==c->k) d=0;  
6 }
```

[출력]:

```
line 3: syntax error: empty parameter name near {  
line 3: syntax error: empty parameter name near {  
line 3: syntax error: empty parameter name near {
```

[설명]: 함수 정의부에서는 abstract_declarator 사용이 가능하지 않아 direct_declarator를 작성해 주어야 한다. 따라서 위의 이미지와 같이 함수 정의부의 int, char, struct l 옆에 parameter name이 존재하지 않다는 error message가 출력된다.

21.

[입력]:

```
1 auto int int int int int int a;
```

[출력]:


```

===== syntax tree =====
N_PROGRAM (0,0)
| (ID="a") TYPE:9b8ce2f0 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | TYPE
| | | (int)

```

[설명]: storage_class로는 AUTO_SYM에 의해서 S_AUTO가 설정된다. 동일한 int_type이 여러 차례 나와도 updateSpecifier 함수에 같은 type_specifier로 인식되어 하나의 int로 결정된다.

22.

[입력]:

```
1 auto int int int int int float a[25]();
```

[출력]:

```

line 1: syntax error: illegal declaration specifiers near a
line 1: syntax error: illegal declaration specifiers near a
line 1: syntax error: illegal declaration specifiers near a
line 1: syntax error: illegal declaration specifiers near a
line 1: syntax error: illegal declaration specifiers near a

```

[설명]: 위의 direct_declarator 'a'에서는 배열('[25]') -> 함수('(')) -> float -> int -> int -> int -> int -> auto 순으로 type을 확인한다. 이때, float과 int가 updateSpecifier에서 다른 type_specifier로 인식이 되어 error message가 출력된다. float과 int가 충돌하는 것이기 때문에 float부터 int가 나오는 부분까지 총 5개의 error message가 출력된다.

23.

[입력]:

```
1 static typedef auto a[25][3]();
```

[출력]:

```

line 1: syntax error: illegal declaration specifiers near a
line 1: syntax error: illegal declaration specifiers near a

```

[설명]: 여러 개의 storage_class가 나오면 updateSpecifier에서 같은 storage_class인지를 확인하는데, 서로 다른 종류의 storage_class로 인해 error message가 출력된다. type은 배열('[25]') -> 배열('[3]') -> 함수('(')) -> auto -> typedef -> static 순으로 검사한다. 이 때, auto부터 static까지 동일한 storage_class가 3개 발생해 error message가 2개 발생하게 된다.

24.

[입력]:

```

1 static int int int int * hamsoo(int a, float b[]){
2     if(b[3]==a)return a;
3 }

```

[출력]:

```
line 1: syntax error: illegal function specifiers near {
```

[설명]: setFunctionDeclaratorSpecifier에서는 함수의 정의 부분에 storage_class가 존재하는지를 확인하고 만약 존재하면 error message를 출력한다.

25.

[입력]:

```
1 auto auto static int int int int * hamsoo(int a, float b[]){
2     if(b[3]==a)return a;
3 }
```

[출력]:

```
line 1: syntax error: illegal declaration specifiers near *
line 1: syntax error: illegal declaration specifiers near *
line 1: syntax error: illegal function specifiers near {
```

[설명]: static -> auto -> auto 순으로 storage_class를 확인하는데, 서로 다른 storage_class인 static과 auto에서 첫 번째 error message가 발생하고, 다음 auto에서 다시 동일한 error message가 발생한다. 마지막 error message는 함수에 존재할 수 없는 storage_class가 존재해 발생하는 error message이다. 이는 updateSpecifier가 여러 차례 반복된 이후에 setFunctionDeclaratorSpecifier로 function의 타입이 맞는지를 확인하기 때문에 위의 이미지와 같은 결과의 순서로 나오게 된다.

26.

[입력]:

```
1 void hamsoo(int, float, float);
2
3 void hamsoo(int a, float b){
4     if(b[3]==a)return a;
5 }
```

[출력]:

```
line 3: syntax error: conflicting parm type in prototype function hamsoo near {
```

[설명]: 함수의 파라미터의 개수가 다른 것 역시도 parameter의 type이 다른 것으로 판단된다.

27.

[입력]:

```
1 void hamsoo(int, char);
2
3 int* hamsoo(int a, char b){
4     if(b[3]==a)return a;
5 }
```

[출력]:

```
line 3: syntax error: illegal or conflicting return type in function hamsoo near {
```

[설명]: 프로토타입(선언부)과 정의부에서의 return type이 달라 해당 error message가 발생하게 된다.

28.

[입력]:

```
1 void* hamsoo(int, char);
2
3 void* hamsoo(int a, char b){
4     if(b[3]==a)return a;
5 }
```

[출력]:

```
***** syntax tree *****
N_PROGRAM (0,0)
| (ID="hamsoo") TYPE:d53eaa0 KIND:FUNC SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | | (ID="") TYPE:d53e52f0 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | (int)
| | | | | (ID="") TYPE:d53e5410 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | (char 1)
| | | | TYPE
| | | | | POINTER
| | | | | ELEMENT_TYPE
| | | | | (void)| (ID="hamsoo") TYPE:d53eacb0 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | TYPE
| | | | FUNCTION
| | | | | PARAMETER
| | | | | | (ID="a") TYPE:d53e52f0 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | | TYPE
| | | | | | | (int)
| | | | | | (ID="b") TYPE:d53e5410 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | | TYPE
| | | | | | | (char 1)
| | | | | TYPE
| | | | | | POINTER
| | | | | | ELEMENT_TYPE
| | | | | | | (void)| | | BODY
| | | | | | | N_STMT_COMPOUND (0,0)
| | | | | | | | N_STMT_LIST (0,0)
| | | | | | | | | N_STMT_IF (0,0)
| | | | | | | | | | N_EXP_EQ (0,0)
| | | | | | | | | | | N_EXP_ARRAY (0,0)
| | | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | | (ID="b") TYPE:d53e5410 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | | | | | | | | N_EXP_INT_CONST (0,0)
| | | | | | | | | | | | | | 3
| | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | (ID="a") TYPE:d53e52f0 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | | | | | N_STMT_RETURN (0,0)
| | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | (ID="a") TYPE:d53e52f0 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | | | N_STMT_LIST_NIL (0,0)
```

[설명]: 함수의 파라미터 수, 타입, 리턴 타입이 동일하고, storage_class를 가지고 있지 않은 함수여서 정상적으로 syntax tree가 출력된다. 함수의 파라미터는 소괄호 안에서 level이 증가하고 감소함으로 각각 level=1이고 kind=PARM(parameter)이다. 또한, 함수 내부의 코드 역시도 적절한 노드와 syntax tree, symbol table이 연결된 것이 확인된다. return a는 int type이긴 하지만 문법적인 오류가 발생하지는 않는다.

29.

[입력]:

```
1 int main(){
2     A x;
3 }
```

[출력]:

```
line 2: syntax error: undefined identifier A near A
line 2: syntax error near x
```

[설명]: 정의되지 않은 type으로 선언하여 “undefined type for identifier” error message를 출력하고자 하였지만, 위와 같은 결과가 발생한다. 이는 A를 IDENTIFIER로 인식하여 발생한 결과이다.

30.

[입력]:

```
1 struct st;
2
3 int main(){
4     struct st new1;
5     struct st { char c; } new2;
6 }
```

[출력]:

```
line 8: syntax error: incomplete forward reference for identifier st at end
```

[설명]: st(struct)가 같은 level에서 선언되고 정의된 것이 아니라 전방 참조와 관련된 error message가 출력되는 것을 확인 할 수 있다.

31.

[입력]:

```
1 union st1;
2
3 int main(){
4     union st2 new1;
5     union st1 { union st2 *n; } new2;
6 }
```

[출력]:

```
line 6: syntax error: incomplete forward reference for identifier st2 near }
line 9: syntax error: incomplete forward reference for identifier st1 at end
```

[설명]: 이전 결과와 동일하게 union 타입의 st1도 같은 level에서 선언되고 정의되어야 한다. 또한, 출력된 결과 순서를 살펴보면, st2, st1 순으로 전방 참조를 하고 있다. 이는 해당 scope 내에서 선언되고 정의까지 되었는지를 확인한다는 것을 알 수 있다.

32.

[입력]:

```
1 struct st1;
2
3 int main(){
4     struct st2;
5     struct st2 { struct st2 *n; } new1;
6     for(;;){
7         struct st3;
8         struct st3 { int *n; } new2;
9     }
10 }
11
12 struct st1 { int a; int b; int c; };
```

[출력]:

```
***** syntax tree *****
N_PROGRAM (0,0)
| (ID="") TYPE:59F3B95B KIND:VMR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | TYPE
| | | STRUCT
| | | | FIELD
| | | | | (ID="a") TYPE:59F3B2F8 KIND:FIELD SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | (int)
| | | | | (ID="b") TYPE:59F3B2F8 KIND:FIELD SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | (int)
| | | | | (ID="c") TYPE:59F3B2F8 KIND:FIELD SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | (int)
| (ID="main") TYPE:59F3Baa0 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | | TYPE
| | | | | (int)
| | | | BODY
| | | | | N_STMT_COMPOUND (0,0)
| | | | | | (ID="") TYPE:59F3Bb00 KIND:VMR SPEC=AUTO LEV=1 VAL=0 ADDR=0
| | | | | | | TYPE
| | | | | | | STRUCT
| | | | | | | | FIELD
| | | | | | | | | (ID="n") TYPE:59F3Bc20 KIND:FIELD SPEC=NULL LEV=2 VAL=0 ADDR=0
| | | | | | | | | | TYPE
| | | | | | | | | | POINTER
| | | | | | | | | | ELEMENT_TYPE
| | | | | | | | | | (DONE:59F3Bb00)
| | | | | | | (ID="new1") TYPE:59F3Bb00 KIND:VMR SPEC=AUTO LEV=1 VAL=0 ADDR=0
| | | | | | | | TYPE
| | | | | | | | (DONE:59F3Bb00)
| | | | | | | | N_STMT_LIST (0,0)
| | | | | | | | | N_STMT_FOR (0,0)
| | | | | | | | | | N_FOR_EXP (0,0)
| | | | | | | | | | | N_STMT_COMPOUND (0,0)
| | | | | | | | | | | | (ID="") TYPE:59F3Bda0 KIND:VMR SPEC=AUTO LEV=2 VAL=0 ADDR=0
| | | | | | | | | | | | TYPE
| | | | | | | | | | | | STRUCT
| | | | | | | | | | | | | FIELD
| | | | | | | | | | | | | | (ID="n") TYPE:59F3Bec0 KIND:FIELD SPEC=NULL LEV=3 VAL=0 ADDR=0
| | | | | | | | | | | | | | | TYPE
| | | | | | | | | | | | | | | POINTER
| | | | | | | | | | | | | | | ELEMENT_TYPE
| | | | | | | | | | | | | | | (int)
| | | | | | | | | | | | (ID="new2") TYPE:59F3Bda0 KIND:VMR SPEC=AUTO LEV=2 VAL=0 ADDR=0
| | | | | | | | | | | | | TYPE
| | | | | | | | | | | | | (DONE:59F3Bda0)
| | | | | | | | | | | | | N_STMT_LIST_NIL (0,0)
| | | | | | | | | | | N_STMT_LIST_NIL (0,0)
| (ID="") TYPE:59F3B95B KIND:VMR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | TYPE
| | | (DONE:59F3B95B)
```

[설명]: struct의 선언부와 정의부가 같은 level에 존재하는 경우 정상적으로 syntax tree가 생성되는 것을 확인할 수 있다.

33.

[입력]:

```
1 union st1;
2
3 int main(){
4     union st2;
5     union st2 { union st2 *n; } new1;
6     for(;;){
7         union st3;
8         union st3 { int *n; } new2;
9     }
10 }
11
12 union st1 { int a; int b; int c; };
```

[출력]:

```
===== syntax tree =====
N_PROGRAM (0,0)
| (ID="") TYPE:44ce8910 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | TYPE
| | | UNION
| | | | FIELD
| | | | | (ID="a") TYPE:44ce32f0 KIND:FIELD SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | | (int)
| | | | | (ID="b") TYPE:44ce32f0 KIND:FIELD SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | | (int)
| | | | | (ID="c") TYPE:44ce32f0 KIND:FIELD SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | | (int)
| | | (ID="main") TYPE:44ce8a00 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | TYPE
| | | | FUNCTION
| | | | | PARAMETER
| | | | | | TYPE
| | | | | | | (int)
| | | | | BODY
| | | | | | N_STMT_COMPOUND (0,0)
| | | | | | | (ID="") TYPE:44ce8b00 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=0
| | | | | | | | TYPE
| | | | | | | | | UNION
| | | | | | | | | | FIELD
| | | | | | | | | | | (ID="n") TYPE:44ce8c20 KIND:FIELD SPEC=NULL LEV=2 VAL=0 ADDR=0
| | | | | | | | | | | | TYPE
| | | | | | | | | | | | | POINTER
| | | | | | | | | | | | | | ELEMENT_TYPE
| | | | | | | | | | | | | | (DONE:44ce8b00)
| | | | | | | | | | | (ID="new1") TYPE:44ce8b00 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=0
| | | | | | | | | | | | TYPE
| | | | | | | | | | | | (DONE:44ce8b00)
| | | | | | | | | | | N_STMT_LIST (0,0)
| | | | | | | | | | | | N_STMT_FOR (0,0)
| | | | | | | | | | | | | N_FOR_EXP (0,0)
| | | | | | | | | | | | | N_STMT_COMPOUND (0,0)
| | | | | | | | | | | | | | (ID="") TYPE:44ce8da0 KIND:VAR SPEC=AUTO LEV=2 VAL=0 ADDR=0
| | | | | | | | | | | | | | TYPE
| | | | | | | | | | | | | | | UNION
| | | | | | | | | | | | | | | | FIELD
| | | | | | | | | | | | | | | | | (ID="n") TYPE:44ce8ec0 KIND:FIELD SPEC=NULL LEV=3 VAL=0 ADDR=0
| | | | | | | | | | | | | | | | | | TYPE
| | | | | | | | | | | | | | | | | | | POINTER
| | | | | | | | | | | | | | | | | | | | ELEMENT_TYPE
| | | | | | | | | | | | | | | | | | | | (int)
| | | | | | | | | | | | | | | | (ID="new2") TYPE:44ce8da0 KIND:VAR SPEC=AUTO LEV=2 VAL=0 ADDR=0
| | | | | | | | | | | | | | | | | TYPE
| | | | | | | | | | | | | | | | | | (DONE:44ce8da0)
| | | | | | | | | | | | | | | | | N_STMT_LIST_NIL (0,0)
| | | | | | | | | | | | | | | | | N_STMT_LIST_NIL (0,0)
| | | (ID="") TYPE:44ce8910 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | | TYPE
| | | | (DONE:44ce8910)
```


[설명]: 이전의 결과와 동일하게 같은 level에서 선언되고 정의된 union들로 인해 정상적으로 syntax tree가 생성된 것을 확인할 수 있다.

34.

[입력]:

```
1 enum e1;
2
3 main(int re){
4     enum e2;
5     if(re==ee.a){
6         enum e3;
7     }
8 }
9 enum e1 { e,f,g };
```

[출력]:

```
line 5: syntax error: undefined identifier ee near ee
line 7: syntax error: incomplete forward reference for identifier e3 near }
line 8: syntax error: incomplete forward reference for identifier e2 near }
```

[설명]: ee라는 enum은 선언하지 않았기 때문에 첫 번째 error message가 출력된다. 이후 scope의 차이로 e3, e2, e1 순서대로 전방 참조가 실행된다. 같은 level에 선언과 정의가 완료되지 않은 e3, e2는 error message가 발생하게 된다.

35.

[입력]:

```
1 enum e1;
2
3 main(int re){
4     enum e1 { a,b,c } ee;
5     if(re==ee.a){
6         enum e2 { d };
7     }
8 }
9 enum e1 { e,f,g };
```

[출력]:

```

===== syntax tree =====
N_PROGRAM (0,0)
| (ID="") TYPE:f2d9f910 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | TYPE
| | | ENUM
| | | | ENUMERATORS
| | | | | (ID="e") TYPE:0 KIND:ENUM_LITERAL SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | | | (ID="f") TYPE:0 KIND:ENUM_LITERAL SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | | | (ID="g") TYPE:0 KIND:ENUM_LITERAL SPEC=NULL LEV=0 VAL=0 ADDR=0
| | (ID="main") TYPE:f2d9fb10 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | | (ID="re") TYPE:f2d9a2f0 KIND:PARM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | | (int)
| | | | | TYPE
| | | | | | (int)
| | | | BODY
| | | | | N_STMT_COMPOUND (0,0)
| | | | | | (ID="ee") TYPE:f2d9fbc0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=0
| | | | | | | TYPE
| | | | | | | | ENUM
| | | | | | | | | ENUMERATORS
| | | | | | | | | | (ID="a") TYPE:0 KIND:ENUM_LITERAL SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | | | | | (ID="b") TYPE:0 KIND:ENUM_LITERAL SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | | | | | (ID="c") TYPE:0 KIND:ENUM_LITERAL SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | | N_STMT_LIST (0,0)
| | | | | | | | N_STMT_IF (0,0)
| | | | | | | | | N_EXP_EQL (0,0)
| | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | (ID="re") TYPE:f2d9a2f0 KIND:PARM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | | | | | N_EXP_STRUCT (0,0)
| | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | (ID="ee") TYPE:f2d9fbc0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=0
| | | | | | | | | | | | a
| | | | | | | | N_STMT_COMPOUND (0,0)
| | | | | | | | | (ID="") TYPE:f2d9ff50 KIND:VAR SPEC=AUTO LEV=2 VAL=0 ADDR=0
| | | | | | | | | | TYPE
| | | | | | | | | | | ENUM
| | | | | | | | | | | | | ENUMERATORS
| | | | | | | | | | | | | | (ID="d") TYPE:0 KIND:ENUM_LITERAL SPEC=NULL LEV=2 VAL=0 ADDR=0
| | | | | | | | | | N_STMT_LIST_NIL (0,0)
| | | | | | | | N_STMT_LIST_NIL (0,0)
| | (ID="") TYPE:f2d9f910 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | TYPE
| | | (DONE:f2d9f910)

```

[설명]: enum 역시도 각 level에 맞춰서 정의하고 선언해야 하는 것을 확인할 수 있다. 또한, level이 다르면 같은 tag의 enum(e1)이 재정의 되는 것을 확인할 수 있다. 기존의 C언어는 전역 타입에 선언된 enum을 수정할 수 없지만, 현재의 프로그램은 가능하게 되어있다.

36.

[입력]:

```
1 enum e1 { a,b,c } new1;
2
3 main(){
4     enum e2 { d,e,f } new2;
5     if(new1.a){
6         new2=new1;
7     }
8 }
```

[출력]:

```
***** syntax tree *****
N_PROGRAM (0,0)
| (ID="new1") TYPE:66ece960 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | TYPE
| | | ENUM
| | | | ENUMERATORS
| | | | | (ID="a") TYPE:0 KIND:ENUM_LITERAL SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | | | (ID="b") TYPE:0 KIND:ENUM_LITERAL SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | | | (ID="c") TYPE:0 KIND:ENUM_LITERAL SPEC=NULL LEV=0 VAL=0 ADDR=0
| (ID="main") TYPE:66ecebf0 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | | TYPE
| | | | | (int)
| | | | BODY
| | | | | N_STMT_COMPOUND (0,0)
| | | | | | (ID="new2") TYPE:66ececc0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=0
| | | | | | | TYPE
| | | | | | | | ENUM
| | | | | | | | | ENUMERATORS
| | | | | | | | | | (ID="d") TYPE:0 KIND:ENUM_LITERAL SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | | | | | (ID="e") TYPE:0 KIND:ENUM_LITERAL SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | | | | | (ID="f") TYPE:0 KIND:ENUM_LITERAL SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | | N_STMT_LIST (0,0)
| | | | | | | | N_STMT_IF (0,0)
| | | | | | | | | N_EXP_STRUCT (0,0)
| | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | (ID="new1") TYPE:66ece960 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | | | | | | | | | | a
| | | | | | | | N_STMT_COMPOUND (0,0)
| | | | | | | | | N_STMT_LIST (0,0)
| | | | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | | | N_EXP_ASSIGN (0,0)
| | | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | | (ID="new2") TYPE:66ececc0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=0
| | | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | | (ID="new1") TYPE:66ece960 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | | | | | | | | | | N_STMT_LIST_NIL (0,0)
| | | | | | | | N_STMT_LIST_NIL (0,0)
```

[설명]: 타입이 다른 enum type의 e1, e2가 ASSIGN 연산으로 값이 대입되는 것을 syntax tree를 확인할 수 있다. 또한, new1이 아닌 new1.a 로 DOT 연산이 가능한 것을 확인할 수 있다.

37.

[입력]:

```
1 struct s1 {int a; float b; char c; } new1;
2
3 main(){
4     struct s2 { int d; float e; char f; } new2;
5     new2=new1;
6 }
```

[출력]:

```
===== syntax tree =====
N_PROGRAM (0,0)
| (ID="new1") TYPE:6fca9960 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | TYPE
| | | STRUCT
| | | | FIELD
| | | | | (ID="a") TYPE:6fca42f0 KIND:FIELD SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | | (int)
| | | | | (ID="b") TYPE:6fca4380 KIND:FIELD SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | | (float)
| | | | | (ID="c") TYPE:6fca4410 KIND:FIELD SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | | (char 1)
| (ID="main") TYPE:6fca9b0 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | | TYPE
| | | | | | (int)
| | | | | BODY
| | | | | N_STMT_COMPOUND (0,0)
| | | | | | (ID="new2") TYPE:6fca9cc0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=0
| | | | | | | TYPE
| | | | | | | | STRUCT
| | | | | | | | | FIELD
| | | | | | | | | | (ID="d") TYPE:6fca42f0 KIND:FIELD SPEC=NULL LEV=2 VAL=0 ADDR=0
| | | | | | | | | | | TYPE
| | | | | | | | | | | | (int)
| | | | | | | | | | (ID="e") TYPE:6fca4380 KIND:FIELD SPEC=NULL LEV=2 VAL=0 ADDR=0
| | | | | | | | | | | TYPE
| | | | | | | | | | | | (float)
| | | | | | | | | | (ID="f") TYPE:6fca4410 KIND:FIELD SPEC=NULL LEV=2 VAL=0 ADDR=0
| | | | | | | | | | | TYPE
| | | | | | | | | | | | (char 1)
| | | | | | | | N_STMT_LIST (0,0)
| | | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | | N_EXP_ASSIGN (0,0)
| | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | (ID="new2") TYPE:6fca9cc0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=0
| | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | (ID="new1") TYPE:6fca9960 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | | | | | | | N_STMT_LIST_NIL (0,0)
```

[설명]: 동일한 개수와 타입의 field를 가진 다른 struct type의 new1과 new2가 ASSIGN 연산으로 연산

이 된 것을 확인할 수 있다. 연산을 위해 여러 노드와 symbol table, type table이 연결된 것을 syntax tree를 통해 확인할 수 있다.

38.

[입력]:

```
1 struct s1 {int a; float b; char c; } new1;
2
3 main(){
4     struct s2 { int d; int e; } new2;
5     new2=new1;
6 }
```

[출력]:

```
===== syntax tree =====
N_PROGRAM (0,0)
| (ID="new1") TYPE:e4d12960 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | TYPE
| | | STRUCT
| | | | FIELD
| | | | | (ID="a") TYPE:e4d0d2f0 KIND:FIELD SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | | (int)
| | | | | (ID="b") TYPE:e4d0d380 KIND:FIELD SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | | (float)
| | | | | (ID="c") TYPE:e4d0d410 KIND:FIELD SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | | (char 1)
| (ID="main") TYPE:e4d12bf0 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | | TYPE
| | | | | | (int)
| | | | | BODY
| | | | | | N_STMT_COMPOUND (0,0)
| | | | | | | (ID="new2") TYPE:e4d12cc0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=0
| | | | | | | | TYPE
| | | | | | | | | STRUCT
| | | | | | | | | | FIELD
| | | | | | | | | | | (ID="d") TYPE:e4d0d2f0 KIND:FIELD SPEC=NULL LEV=2 VAL=0 ADDR=0
| | | | | | | | | | | | TYPE
| | | | | | | | | | | | | (int)
| | | | | | | | | | | (ID="e") TYPE:e4d0d2f0 KIND:FIELD SPEC=NULL LEV=2 VAL=0 ADDR=0
| | | | | | | | | | | | TYPE
| | | | | | | | | | | | | (int)
| | | | | | | | | N_STMT_LIST (0,0)
| | | | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | | | N_EXP_ASSIGN (0,0)
| | | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | | (ID="new2") TYPE:e4d12cc0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=0
| | | | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | | | (ID="new1") TYPE:e4d12960 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | | | | | | | | | | | | | N_STMT_LIST_NIL (0,0)
```

[설명]: 다른 개수와 type의 field를 가진 struct type new1, new2가 ASSIGN 연산이 되는 것을 syntax tree를 통해 확인할 수 있다. expression을 위한 노드와 symbol table이 연결되었다.

39.

[입력]:

```
1 union s1 {int a; float b; char c; } new1;
2
3 main(){
4     union s2 { int d; int e; } new2;
5     new2=new1;
6 }
```

[출력]:

```
***** syntax tree *****
N_PROGRAM (0,0)
| (ID="new1") TYPE:4a0f2960 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | TYPE
| | | UNION
| | | | FIELD
| | | | | (ID="a") TYPE:4a0ed2f0 KIND:FIELD SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | | (int)
| | | | | (ID="b") TYPE:4a0ed380 KIND:FIELD SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | | (float)
| | | | | (ID="c") TYPE:4a0ed410 KIND:FIELD SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | | (char 1)
| (ID="main") TYPE:4a0f2b70 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | | TYPE
| | | | | | (int)
| | | | | BODY
| | | | | | N_STMT_COMPOUND (0,0)
| | | | | | | (ID="new2") TYPE:4a0f2cc0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=0
| | | | | | | | TYPE
| | | | | | | | | UNION
| | | | | | | | | | FIELD
| | | | | | | | | | | (ID="d") TYPE:4a0ed2f0 KIND:FIELD SPEC=NULL LEV=2 VAL=0 ADDR=0
| | | | | | | | | | | | TYPE
| | | | | | | | | | | | | (int)
| | | | | | | | | | | (ID="e") TYPE:4a0ed2f0 KIND:FIELD SPEC=NULL LEV=2 VAL=0 ADDR=0
| | | | | | | | | | | | TYPE
| | | | | | | | | | | | | (int)
| | | | | | | | N_STMT_LIST (0,0)
| | | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | | N_EXP_ASSIGN (0,0)
| | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | (ID="new2") TYPE:4a0f2cc0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=0
| | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | (ID="new1") TYPE:4a0f2960 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | | | | | | | N_STMT_LIST NIL (0,0)
```

[설명]: 이전 struct의 결과와 동일하게 union 역시도 다른 개수와 타입의 field를 가진 IDENTIFIER가 ASSIGN 연산이 가능한 것을 확인할 수 있다.

40.

[입력]:

```
1 int hamsoo(int a);
```


[출력]:

```
===== syntax tree =====
N_PROGRAM (0,0)
| (ID="hamsoo") TYPE:191dda10 KIND:FUNC SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | | (ID="a") TYPE:191d82f0 KIND:PARM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | | (int)
| | | | | TYPE
| | | | | (int)
```

[설명]: compound_statement가 없는 함수는 이후에 완전한 정의를 하지 않아도 error message가 발생하지 않는다. syntax tree를 통해 함수의 타입, 파라미터의 타입, level, kind가 제대로 구성된 것을 확인할 수 있다.

41.

[입력]:

```
1 int hamsoo(int a,...){
2     switch(a){
3         case 1: hamsoo(1,2); break;
4         case 2: hamsoo(2,3); continue;
5         default: return 90+(123++)*4;
6     }
7 }
```

[출력]:

```
***** syntax tree *****
N_PROGRAM (0,0)
| (ID="hansoo") TYPE:32f6ca68 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | | (ID="a") TYPE:32f672f0 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | (int)
| | | | | (ID="") TYPE:0 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | (int)
| | | | BODY
| | | | | N_STMT_COMPOUND (0,0)
| | | | | | N_STMT_LIST (0,0)
| | | | | | | N_STMT_SWITCH (0,0)
| | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | (ID="a") TYPE:32f672f0 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | | | N_STMT_COMPOUND (0,0)
| | | | | | | | | N_STMT_LIST (0,0)
| | | | | | | | | | N_STMT_LABEL_CASE (0,0)
| | | | | | | | | | | N_EXP_INT_CONST (0,0)
| | | | | | | | | | | | 1
| | | | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | | | N_EXP_FUNCTION_CALL (0,0)
| | | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | | (ID="hansoo") TYPE:32f6ca68 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | | | | | | | | | | N_ARG_LIST (0,0)
| | | | | | | | | | | | | N_EXP_INT_CONST (0,0)
| | | | | | | | | | | | | | 1
| | | | | | | | | | | | | N_ARG_LIST (0,0)
| | | | | | | | | | | | | | N_EXP_INT_CONST (0,0)
| | | | | | | | | | | | | | | 2
| | | | | | | | | | | | | N_ARG_LIST_NIL (0,0)
| | | | | | | | N_STMT_LIST (0,0)
| | | | | | | | | N_STMT_BREAK (0,0)
| | | | | | | | N_STMT_LIST (0,0)
| | | | | | | | | N_STMT_LABEL_CASE (0,0)
| | | | | | | | | | N_EXP_INT_CONST (0,0)
| | | | | | | | | | | 2
| | | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | | N_EXP_FUNCTION_CALL (0,0)
| | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | (ID="hansoo") TYPE:32f6ca68 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | | | | | | | | | N_ARG_LIST (0,0)
| | | | | | | | | | | | N_EXP_INT_CONST (0,0)
| | | | | | | | | | | | | 2
| | | | | | | | | | | | N_ARG_LIST (0,0)
| | | | | | | | | | | | | N_EXP_INT_CONST (0,0)
| | | | | | | | | | | | | | 3
| | | | | | | | | | | | N_ARG_LIST_NIL (0,0)
| | | | | | | | N_STMT_LIST (0,0)
| | | | | | | | | N_STMT_CONTINUE (0,0)
| | | | | | | | N_STMT_LIST (0,0)
| | | | | | | | | N_STMT_LABEL_DEFAULT (0,0)
| | | | | | | | | | N_STMT_RETURN (0,0)
| | | | | | | | | | | N_EXP_ADD (0,0)
| | | | | | | | | | | | N_EXP_INT_CONST (0,0)
| | | | | | | | | | | | | 90
| | | | | | | | | | | | N_EXP_MUL (0,0)
| | | | | | | | | | | | | N_EXP_POST_INC (0,0)
| | | | | | | | | | | | | N_EXP_INT_CONST (0,0)
| | | | | | | | | | | | | | 123
| | | | | | | | | | | | N_EXP_INT_CONST (0,0)
| | | | | | | | | | | | | 4
| | | | | | | | N_STMT_LIST_NIL (0,0)
| | | | | N_STMT_LIST_NIL (0,0)
```

[설명]: 함수의 파라미터로 ‘...’이 입력되면 makeDummyIdentifier에 의해 이름이 없는 Identifier를 만들어 parameter로 연결한다. 그리고 함수의 body 내부의 switch statement node에서는, LABEL_CASE 노드에 해당 case로 설정되는 INT_CONSTANT와 expression, jump_statement가 적절하게 연결되는 것

을 확인할 수 있다. expression인 function_call에서는 왼쪽->오른쪽 순으로 파라미터가 선언된다. 또한, INT_CONSTANT는 N_EXP_INT_CONST라는 노드 하위에 실제 값이 연결되어 있다. default label도 적절한 출력되었고, 수식도 순서에 맞게 출력되었다.

42.

[입력]:

```
1 int hamsoo(int a,...){
2     switch(a){
3         int b;
4         case 1: hamsoo(b=a+a*a, b ); break;
5     }
6 }
```

[출력]:

```
***** syntax tree *****
N_PROGRAM (0,0)
| ((ID="hamsoo") TYPE:30314a00 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | | ((ID="a") TYPE:3030f2f0 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | | (int)
| | | | | ((ID="") TYPE:0 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | | (int)
| | | | | BODY
| | | | | | N_STMT_COMPOUND (0,0)
| | | | | | | N_STMT_LIST (0,0)
| | | | | | | | N_STMT_SWITCH (0,0)
| | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | ((ID="a") TYPE:3030f2f0 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | | | | | | N_STMT_COMPOUND (0,0)
| | | | | | | | | | | | ((ID="b") TYPE:3030f2f0 KIND:VAR SPEC=AUTO LEV=2 VAL=0 ADDR=0
| | | | | | | | | | | | | TYPE
| | | | | | | | | | | | | | (int)
| | | | | | | | | | | | | | N_STMT_LIST (0,0)
| | | | | | | | | | | | | | | N_STMT_LABEL_CASE (0,0)
| | | | | | | | | | | | | | | | N_EXP_INT_CONST (0,0)
| | | | | | | | | | | | | | | | | 1
| | | | | | | | | | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | | | | | | | | | N_EXP_FUNCTION_CALL (0,0)
| | | | | | | | | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | | | | | | | | ((ID="hamsoo") TYPE:30314a00 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | | | | | | | | | | | | | | | | | N_ARG_LIST (0,0)
| | | | | | | | | | | | | | | | | | | | N_EXP_ASSIGN (0,0)
| | | | | | | | | | | | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | | | | | | | | | | | ((ID="b") TYPE:3030f2f0 KIND:VAR SPEC=AUTO LEV=2 VAL=0 ADDR=0
| | | | | | | | | | | | | | | | | | | | | | N_EXP_ADD (0,0)
| | | | | | | | | | | | | | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | | | | | | | | | | | | | ((ID="a") TYPE:3030f2f0 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | | | | | | | | | | | | | | | | | | | | N_EXP_MUL (0,0)
| | | | | | | | | | | | | | | | | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | | | | | | | | | | | | | | | | ((ID="a") TYPE:3030f2f0 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | | | | | | | | | | | | | | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | ((ID="a") TYPE:3030f2f0 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | N_ARG_LIST (0,0)
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | ((ID="b") TYPE:3030f2f0 KIND:VAR SPEC=AUTO LEV=2 VAL=0 ADDR=0
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | N_ARG_LIST_NIL (0,0)
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | N_STMT_LIST (0,0)
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | N_STMT_BREAK (0,0)
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | N_STMT_LIST_NIL (0,0)
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | N_STMT_LIST_NIL (0,0)
```

설명: 연산이 이루어진 변수 b가 함수의 파라미터로 입력되어 syntax tree에 연결되는 것을 확인할 수 있다. 즉, 연산이 먼저 수행되어 해당 결과가 파라미터로 전달된다. 또한, b의 타입을 char, float으로 변경하여 연산 타입이 맞지 않더라도 syntax tree가 제대로 출력된다.

43.

[입력]:

```
1 int hamsoo(int a,...){
2     switch(a){
3         case 1: hamsoo(int b=a+a*a, b ); break;
4     }
5 }
```

[출력]:

```
line 3: syntax error near int
```

[설명]: 함수 파라미터 내부에 변수를 선언하는 것은 error message가 출력된다. 문법적으로 정의되지 않은 경우이기 때문이다.

44.

[입력]:

```
1 int hamsoo(int a,...){
2     if(a[2]->x){ return a[2]->x; }
3     else { return a[2]->y; }
4 }
```

[출력]:

```
***** Syntax Tree *****
N_PROGRAM (0,0)
| (ID="hamsoo") TYPE:1f015a60 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | | (ID="a") TYPE:1f0102f0 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | (int)
| | | | | (ID="") TYPE:0 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | (int)
| | | | BODY
| | | | | N_STMT_COMPOUND (0,0)
| | | | | | N_STMT_LIST (0,0)
| | | | | | | N_STMT_IF_ELSE (0,0)
| | | | | | | | N_EXP_ARRAY (0,0)
| | | | | | | | | N_EXP_ARRAY (0,0)
| | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | (ID="a") TYPE:1f0102f0 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | | | | | N_EXP_INT_CONST (0,0)
| | | | | | | | | | | 2
| | | | | | | | | | | x
| | | | | | | | N_STMT_COMPOUND (0,0)
| | | | | | | | | N_STMT_LIST (0,0)
| | | | | | | | | | N_STMT_RETURN (0,0)
| | | | | | | | | | | N_EXP_ARRAY (0,0)
| | | | | | | | | | | | N_EXP_ARRAY (0,0)
| | | | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | | (ID="a") TYPE:1f0102f0 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | | | | | | | | N_EXP_INT_CONST (0,0)
| | | | | | | | | | | | | | 2
| | | | | | | | | | | | | | x
| | | | | | | | | | N_STMT_LIST_NIL (0,0)
| | | | | | N_STMT_COMPOUND (0,0)
| | | | | | | N_STMT_LIST (0,0)
| | | | | | | | N_STMT_RETURN (0,0)
| | | | | | | | | N_EXP_ARRAY (0,0)
| | | | | | | | | | N_EXP_ARRAY (0,0)
| | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | (ID="a") TYPE:1f0102f0 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | | | | | | N_EXP_INT_CONST (0,0)
| | | | | | | | | | | | 2
| | | | | | | | | | | | x
| | | | | | | | N_STMT_LIST_NIL (0,0)
| | | | | N_STMT_LIST_NIL (0,0)
```

[설명]: IF_ELSE statement가 node를 통해 syntax tree에 연결된 것을 확인할 수 있다. expression과 compound_statement 내부에서의 expression이 연산자 우선순위에 맞게 잘 연결되어 있고, 배열의 크기 역시도 parameter에 잘 연결되어 있다. 또한, return statement 역시도 node로 연결되어 있는 것을 확인할 수 있다. 하지만 정의되어 있지 않은 identifier(x, y)를 판단하지 못하고 있다.

45.

[입력]:

```
1 hamsoo(){
2     int a=(((("bye")))+ "hl"+81);
3 }
```

[출력]:

```
===== syntax tree =====
N_PROGRAM (0,0)
| (ID="hamsoo") TYPE:91d20960 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | | TYPE
| | | | | (int)
| | | | BODY
| | | | | N_STMT_COMPOUND (0,0)
| | | | | | (ID="a") TYPE:91d1b2f0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | (int)
| | | | | | INIT
| | | | | | | N_INIT_LIST_ONE (0,0)
| | | | | | | | N_EXP_ADD (0,0)
| | | | | | | | | N_EXP_ADD (0,0)
| | | | | | | | | | N_EXP_STRING_LITERAL (0,0)
| | | | | | | | | | | "bye"
| | | | | | | | | | | N_EXP_STRING_LITERAL (0,0)
| | | | | | | | | | | | "hl"
| | | | | | | | | | N_EXP_AMP (0,0)
| | | | | | | | | | | N_EXP_INT_CONST (0,0)
| | | | | | | | | | | | 1
| | | | | | | N_STMT_LIST_NIL (0,0)
```

[설명]: LP expression RP는 단순히 primary_expression으로 reduce되기 때문에, syntax_tree에는 따로 노드가 추가되지는 않는다. 그리고 STRING_LITERAL과의 연산과 AMP 연산이 수행된 INT_CONST의 값이 더해지는 과정이 제대로 출력되었다. 이 때, 문자열과 숫자는 노드 하위에 존재한다.

46.

[입력]:

```
1 int hamsoo(int a,char b){
2     if(a){
3         if(b){
4             for(b=0;b<=3;--b){
5                 hamsoo(1,2);
6             }
7         }
8     }
9 }
```

[출력]:

```
===== syntax tree =====
N_PROGRAM (0,0)
| (ID="hamsoo") TYPE:13e6faa0 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | | (ID="a") TYPE:13e6a2f0 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | | (int)
| | | | | | (ID="b") TYPE:13e6a410 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | | | TYPE
| | | | | | | | (char 1)
| | | | | | TYPE
| | | | | | | (int)
| | | | | BODY
| | | | | | N_STMT_COMPOUND (0,0)
| | | | | | | N_STMT_LIST (0,0)
| | | | | | | | N_STMT_IF (0,0)
| | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | (ID="a") TYPE:13e6a2f0 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | | | | | N_STMT_COMPOUND (0,0)
| | | | | | | | | | | N_STMT_LIST (0,0)
| | | | | | | | | | | | N_STMT_IF (0,0)
| | | | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | | | (ID="b") TYPE:13e6a410 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | | | | | | | | | N_STMT_COMPOUND (0,0)
| | | | | | | | | | | | | | | N_STMT_LIST (0,0)
| | | | | | | | | | | | | | | | N_STMT_FOR (0,0)
| | | | | | | | | | | | | | | | | N_FOR_EXP (0,0)
| | | | | | | | | | | | | | | | | | N_EXP_ASSIGN (0,0)
| | | | | | | | | | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | | | | | | | | | (ID="b") TYPE:13e6a410 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | | | | | | | | | | | | | | | N_EXP_INT_CONST (0,0)
| | | | | | | | | | | | | | | | | | | | | 0
| | | | | | | | | | | | | | | | | | | | N_EXP_LEQ (0,0)
| | | | | | | | | | | | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | | | | | | | | | | | (ID="b") TYPE:13e6a410 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | | | | | | | | | | | | | | | | | N_EXP_INT_CONST (0,0)
| | | | | | | | | | | | | | | | | | | | | | | 3
| | | | | | | | | | | | | | | | | | | | N_EXP_PRE_DEC (0,0)
| | | | | | | | | | | | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | | | | | | | | | | | (ID="b") TYPE:13e6a410 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | | | | | | | | | | | | | | | | N_STMT_COMPOUND (0,0)
| | | | | | | | | | | | | | | | | | | | | | N_STMT_LIST (0,0)
| | | | | | | | | | | | | | | | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | | | | | | | | | | | | | | | N_EXP_FUNCTION_CALL (0,0)
| | | | | | | | | | | | | | | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | | | | | | | | | | | | | | (ID="hamsoo") TYPE:13e6faa0 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | | | | | | | | | | | | | | | | | | | | | | | | N_ARG_LIST (0,0)
| | | | | | | | | | | | | | | | | | | | | | | | | | N_EXP_INT_CONST (0,0)
| | | | | | | | | | | | | | | | | | | | | | | | | | | 1
| | | | | | | | | | | | | | | | | | | | | | | | | | N_ARG_LIST (0,0)
| | | | | | | | | | | | | | | | | | | | | | | | | | | N_EXP_INT_CONST (0,0)
| | | | | | | | | | | | | | | | | | | | | | | | | | | | 2
| | | | | | | | | | | | | | | | | | | | | | | | | | | N_ARG_LIST_NIL (0,0)
| | | | | | | | | | | | | | | | | | | | | | | | | | | N_STMT_LIST_NIL (0,0)
| | | | | | | | | | | | | | | | | | | | | | | | | | | N_STMT_LIST_NIL (0,0)
| | | | | | | | | | | | | | | | | | | | | | | | | | | N_STMT_LIST_NIL (0,0)
```

[설명]: N_STMT_IF 내부에 N_STMT_IF 가 존재하고 다시 내부에 N_STMT_FOR 가 존재한다. for문의 expression에는 이전에 정의 되어있던 IDENTIFIER b의 연산이 SEMICOLON 2개로 구분되어 있다. 또한, compound_statement에서 다시 N_EXP_FUNCTION_CALL이 이루어지고 있는 것을 확인할 수 있다.

47.

[입력]:

```
1 struct s {int a;} new1;
2 struct s new2;
3 struct s new3;
```

[출력]:

```
***** syntax tree *****
N_PROGRAM (0,0)
| (ID="new1") TYPE:43c92960 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | TYPE
| | | STRUCT
| | | | FIELD
| | | | | (ID="a") TYPE:43c8d2f0 KIND:FIELD SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | | (int)
| (ID="new2") TYPE:43c92960 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | TYPE
| | | (DONE:43c92960)
| (ID="new3") TYPE:43c92960 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | TYPE
| | | (DONE:43c92960)
```

[설명]: 같은 tag의 struct을 가진 new1, new2, new3는 모두 동일한 type, kind, spec을 가진 것을 확인할 수 있다. 또한 같은 scope이기 때문에 level 역시 동일하다.

48.

[입력]:

```
1 int a;
2 struct s { int* b; } new1;
3
4 int main(){
5     int b;
6     b=new1->b;
7 }
```

[출력]:

```
===== syntax tree =====
N_PROGRAM (0,0)
| (ID="a") TYPE:5170a2f0 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | TYPE
| | | (int)
| (ID="new1") TYPE:5170f9f0 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | TYPE
| | | STRUCT
| | | | FIELD
| | | | | (ID="b") TYPE:5170fa50 KIND:FIELD SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | TYPE
| | | | | | POINTER
| | | | | | ELEMENT_TYPE
| | | | | | | (int)
| (ID="main") TYPE:5170fc00 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | TYPE
| | | | | (int)
| | | | BODY
| | | | | N_STMT_COMPOUND (0,0)
| | | | | | (ID="b") TYPE:5170a2f0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=0
| | | | | | TYPE
| | | | | | | (int)
| | | | | | N_STMT_LIST (0,0)
| | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | N_EXP_ASSIGN (0,0)
| | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | (ID="b") TYPE:5170a2f0 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=0
| | | | | | | | | N_EXP_ARROW (0,0)
| | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | (ID="new1") TYPE:5170f9f0 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0
| | | | | | | | | | | b
| | | | | | | N_STMT_LIST_NIL (0,0)
```

[설명]: struct 내부의 b와 main 함수 내부의 b 동일하게 level=1이다. 하지만, 각각 선언이 될 때는 level의 값이 증가하고 감소하면서 current_id의 위치가 달라지기 때문에 중복 선언 문제가 발생하지 않고 적절한 syntax tree가 출력된다.

49.

[입력]:

```
1 int main(){
2     sizeof(++sizeof(!sizeof(int)));
3 }
```

[출력]:

```
===== syntax tree =====
N_PROGRAM (0,0)
| (ID="main") TYPE:5444e980 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | TYPE
| | | | | (int)
| | | | BODY
| | | | | N_STMT_COMPOUND (0,0)
| | | | | | N_STMT_LIST (0,0)
| | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | N_EXP_SIZE_EXP (0,0)
| | | | | | | | | N_EXP_PRE_INC (0,0)
| | | | | | | | | | N_EXP_SIZE_EXP (0,0)
| | | | | | | | | | | N_EXP_NOT (0,0)
| | | | | | | | | | | | N_EXP_SIZE_TYPE (0,0)
| | | | | | | | | | | | | (int)
| | | | | | | | | | | | | N_STMT_LIST NIL (0,0)
```

[설명]: sizeof 소괄호 안에 다시 unary_expression을 허용하기 때문에 sizeof에 여러 연산을 수행할 수 있고, syntax tree를 통해 연산 순서를 확인할 수 있다.

50.

[입력]:

```
1 int main(){
2     sizeof(int) && 3 > 13 >= 3;
3 }
```

[출력]:

```
===== syntax tree =====
N_PROGRAM (0,0)
| (ID="main") TYPE:74038900 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | TYPE
| | | | | (int)
| | | | BODY
| | | | | N_STMT_COMPOUND (0,0)
| | | | | | N_STMT_LIST (0,0)
| | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | N_EXP_AND (0,0)
| | | | | | | | | N_EXP_SIZE_TYPE (0,0)
| | | | | | | | | | (int)
| | | | | | | | | | N_EXP_GEQ (0,0)
| | | | | | | | | | | N_EXP_GTR (0,0)
| | | | | | | | | | | | N_EXP_INT_CONST (0,0)
| | | | | | | | | | | | | 3
| | | | | | | | | | | | N_EXP_NOT (0,0)
| | | | | | | | | | | | | N_EXP_INT_CONST (0,0)
| | | | | | | | | | | | | | 3
| | | | | | | | | | | | | N_EXP_INT_CONST (0,0)
| | | | | | | | | | | | | | 3
| | | | | | | | | | | | N_STMT_LIST NIL (0,0)
```

설명: &&(AMPAMP) 연산, >(GTR) 연산, !(NOT) 연산, >=(GEQ) 연산 모두 연산자 우선 순서에 맞게 syntax tree가 출력된 것을 확인하였다.

51.

[입력]:

```
1 int main(int a, int b){
2     do{
3         a = a || *(--b);
4     }while(!a!=+b->b.a[sizeof(++&+a)]--);
5 }
```

[출력]:

```
===== syntax tree =====
N_PROGRAM (0,0)
|
| (ID="main") TYPE:868e8a0b KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| |
| | TYPE
| | |
| | | FUNCTION
| | | |
| | | | PARAMETER
| | | | |
| | | | | (ID="a") TYPE:868e32f0 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | |
| | | | | | TYPE
| | | | | | |
| | | | | | | (int)
| | | | | |
| | | | | | (ID="b") TYPE:868e32f0 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | |
| | | | | | | TYPE
| | | | | | | |
| | | | | | | | (int)
| | | | | |
| | | | | | TYPE
| | | | | | |
| | | | | | | (int)
| | | | |
| | | | BODY
| | | | |
| | | | | N_STMT_COMPOUND (0,0)
| | | | | |
| | | | | | N_STMT_LIST (0,0)
| | | | | | |
| | | | | | | N_STMT_DO (0,0)
| | | | | | | |
| | | | | | | | N_STMT_COMPOUND (0,0)
| | | | | | | | |
| | | | | | | | | N_STMT_LIST (0,0)
| | | | | | | | | |
| | | | | | | | | | N_STMT_EXPRESSION (0,0)
| | | | | | | | | | |
| | | | | | | | | | | N_EXP_ASSIGN (0,0)
| | | | | | | | | | | |
| | | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | |
| | | | | | | | | | | | | (ID="a") TYPE:868e32f0 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | | | | | | | | N_EXP_OR (0,0)
| | | | | | | | | | | | | |
| | | | | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | (ID="a") TYPE:868e32f0 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | | | | | | | | | | N_EXP_STAR (0,0)
| | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | N_EXP_PRE_DEC (0,0)
| | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | (ID="b") TYPE:868e32f0 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | | | | | | | | | | | | | N_STMT_LIST_NIL (0,0)
| | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | N_EXP_NEQ (0,0)
| | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | N_EXP_NOT (0,0)
| | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | (ID="a") TYPE:868e32f0 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | | | | | | | | | | | | | N_EXP_PLUS (0,0)
| | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | N_EXP_POST_DEC (0,0)
| | | | | | | | | | | | | | | | | | | N_EXP_ARRAY (0,0)
| | | | | | | | | | | | | | | | | | | N_EXP_STRUCT (0,0)
| | | | | | | | | | | | | | | | | | | N_EXP_ARROW (0,0)
| | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | (ID="b") TYPE:868e32f0 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | | | | | | | | | | | | | | | | b
| | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | a
| | | | | | | | | | | | | | | | | | | | | N_EXP_SIZE_EXP (0,0)
| | | | | | | | | | | | | | | | | | | | | N_EXP_PRE_INC (0,0)
| | | | | | | | | | | | | | | | | | | | | N_EXP_AMP (0,0)
| | | | | | | | | | | | | | | | | | | | | N_EXP_PRE_INC (0,0)
| | | | | | | | | | | | | | | | | | | | | N_EXP_IDENT (0,0)
| | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | (ID="a") TYPE:868e32f0 KIND:PARAM SPEC=NULL LEV=1 VAL=0 ADDR=0
| | | | | | | | | | | | | | | | | | | | | | N_STMT_LIST_NIL (0,0)
```

[설명]: do_while의 compound에서 ||(BARBAR) 연산과 *(STAR), --(MINUSMINUS) 연산이 연산 순서에 맞게 출력되었다. 또한, while의 expression에서 !(NOT), !=(NEQ), +(PLUS), ->(ARROW), .(DOT), sizeof, ++(PLUSPLUS), &(AMP), --(MINUSMINUS) 연산이 연산 우선순위에 맞게 syntax tree가 형성된 것을 확인할 수 있다.

52.

[입력]:

```
1 typedef typedef newtypedef;  
2 newtypedef newtypedef newnewtypedef;
```

[출력]:

```
***** syntax tree *****  
N_PROGRAM (0,0)  
| (ID="newtypedef") TYPE:c8d012f0 KIND:TYPE SPEC=TYPEDEF LEV=0 VAL=0 ADDR=0  
| | TYPE  
| | | (int)  
| (ID="newnewtypedef") TYPE:c8d012f0 KIND:VAR SPEC=AUTO LEV=0 VAL=0 ADDR=0  
| | TYPE  
| | | (int)
```

[설명]: 입력의 맨 첫 번째 줄에서 newtypedef의 kind가 type이 되어 type_table에 포함된 것이 확인되지만, 실제 type을 보면 int로 설정이 되어있다. 첫 번째, 두 번째 typedef가 둘 다 TYPEDEF_SYM으로 처리되는데, 이와 같이 처리가 되면 storage_class + storage_class의 형태가 되고, type_specifier가 없기 때문에 setDefaultSpecifier에서 type_specifier의 default 값인 int가 설정된 것으로 예측된다. 또한, newtypedef가 int_type이기 때문에, 결국에 두 번째 줄은 int int newnewtypedef가 되어서 int 타입이 되는 것으로 판단된다.